Explainable Supply Chain Optimization: Reinforcement Learning, Tuned Classical Baselines, and a Number-Checked Language-Model Explainer for Retail Inventory

Santosh K. Koushik
Amrita Vishwa Vidyapeetham, Coimbatore
CB.SC.U4AIE23310 CB.SC.U4AIE23312

Abstract

A retailer must decide every day, for every product, how much to order and from which supplier. We build a complete decision-support system for this problem and evaluate it against tuned classical policies. A simulator calibrated on 1,067,371 real sales records provides the environment. A MaskablePPO agent chooses quantity and supplier jointly. Four textbook policies serve as baselines, each tuned on separate seeds. A language model explains each decision in business English and writes supply-chain crises for stress testing. All results are scored on 30 held-out seeds with paired tests.

The tuned agent earns Rs. 79.7 lakh per 180-day run, which is 90.8% of the best classical policy (Rs. 87.8 lakh) and above two of the four classical policies. The remaining gap is statistically significant (paired $t = -6.45$, $n = 30$, 95% CI [5.5, 10.6] lakh). We trace it to supplier fragmentation: the agent pays about twice the ordering fees, because its observation contains no ordering history. Action masking improves the agent roughly threefold, and hyperparameter tuning closes 38% of what remains. Training on LLM-generated crises does *not* improve robustness (paired $t = +1.69$, not significant), whereas plain tuning does. For the explainer, 5 of 30 raw LLM outputs contained a number the simulator never produced; a grounding check caught all 5, so delivered accuracy is 100% against 83% raw. We report these negative results as findings, because every comparison was designed so that a positive result would have meant something.

1 Introduction

Retail replenishment can fail in two directions that cost different amounts. Ordering too much ties up money, fills a shared warehouse, and leaves seasonal goods unsold. Ordering too little loses the sale for good. Supplier choice adds a second decision: one supplier is cheap but slow, one is fast but dear, and one looks reasonable but ships only 80% of an order and goes offline most often. Because today's order becomes next week's stock, the task is a sequential decision problem.

Classical rules such as (s, S), EOQ with a reorder point, the newsvendor model, and forecast-plus-safety-stock treat each product on its own and assume well-behaved demand. In our data, daily demand has a variance about 216 times its mean, far beyond what a Poisson model allows. Reinforcement learning (RL) can in principle learn a joint ordering and sourcing policy that a formula cannot express, and deep RL has been tried on inventory problems with mixed results against tuned heuristics [3, 5]. A second obstacle is trust: a buyer will not act on a recommendation they cannot interpret.

This paper therefore tests a hypothesis rather than assuming it: that an RL agent can match or beat tuned classical policies, and that an LLM can make its decisions understandable without making things up. We keep the baselines strong, keep the test seeds separate, and report the result whichever way it falls. Our contributions are:

1. An end-to-end system: a calibrated simulator, a MaskablePPO agent with a joint quantity–supplier action, four tuned classical baselines, an LLM explainer, an LLM crisis generator, and a recommend-only dashboard (Sections 3–5).
2. A fair comparison on 30 held-out seeds with paired significance tests, showing that the tuned agent reaches 90.8% of the best classical policy and that the remaining Rs. 8.1 lakh gap is real and traceable to a specific cause (Sections 6.1 and 7).
3. Two controlled ablations: action masking is worth about $3\times$ and tuning closes 38% of the rest, while training on LLM-generated crises does not improve robustness (Sections 6.2 and 6.3).
4. A measurement of the explainer showing that the grounding check is essential: raw accuracy is 83%, delivered accuracy 100% (Section 6.4).
5. A test of whether the demand pattern is stable across the two years of data, answering the concern that the data is old (Section 6.5).

2 Related Work

Deep RL for inventory. Oroojlooyjadid et al. [5] apply a deep Q-network to the beer game and report performance at or above base-stock policies under cooperation, in a serial

chain with a single product and no sourcing choice. Gijsbrechts et al. [3] apply an actor-critic method to lost-sales and dual-sourcing problems and find it competitive with tuned heuristics but not clearly superior, at a high tuning cost. Boute et al. [4] survey the area and name fair benchmarking and interpretability as open problems. Our setting adds joint quantity and supplier selection under one shared warehouse, and we treat fair benchmarking as a design requirement.

Policy optimization and action masking. Proximal Policy Optimization [1] optimizes a clipped surrogate objective and is a stable on-policy default. Huang and Ontañón [2] show that invalid-action masking is a valid policy gradient and scales to large action spaces, which is what makes our $21^{10} \approx 1.7 \times 10^{13}$-option daily action tractable. We use the Stable-Baselines3 ecosystem [7].

Retail demand data. Chen et al. [6] study data mining on online-retail transactions, the same family of data as the UCI Online Retail II set [8] that we calibrate on.

Explanation. Learned policies output a number, not a reason, so they rarely reach a decision-maker. We do not claim to open the network. Instead the LLM only restates facts the simulator computed, and a check rejects any number it invents (Section 4.4). To our knowledge the combination of an RL inventory agent, a number-checked LLM explainer, and an LLM that generates the scenarios used to test it has not been evaluated together; we make no claim beyond this single setting.

3 Problem Setup

Data. UCI Online Retail II [8] holds 1,067,371 sales records from a UK online gift shop, December 2009 to December 2011. Cancelled orders, returns, negative quantities, and missing product codes are removed, and sales are aggregated to one row per product per day. Saturdays are dropped because the shop never traded on them; filling them with zeros would teach a false weekly dip. We select ten products on purpose to span three demand types: four steady (coefficient of variation 1.02–1.20), three moderate (1.29–1.65), and three bursty (2.09–2.34). Only data before 1 June 2011 builds the simulator. The agent never sees the records: we measure mean demand, variability, weekday pattern, month pattern, and zero-sales frequency, and the simulator reproduces them with a Gamma–Poisson (negative binomial) demand model.

A rejected alternative. A newer Kaggle retail-inventory dataset (2022–2024) has no weekday and no seasonal pattern and is synthetic; we rejected it. Newer but synthetic is worse than older but real for calibrating demand shape.

Supplier	Price	Lead	Ships	Fee
EconoSource	base	5–8 d	90%	Rs. 2,100
RapidTrade	+35%	1–3 d	98%	Rs. 4,725
MidWay	+15%	3–5 d	80%	Rs. 3,150

Table 1: The three suppliers. Minimum orders are 100, 20, and 50 units. MidWay goes offline most often, RapidTrade almost never.

Environment. The environment is a Gymnasium simulator of 10 products, 3 suppliers, and 180 days per run, with a 14-day warm-up that is not scored. All products share one warehouse of 20,000 units; deliveries beyond capacity are refused and written off. Each day proceeds in order: deliveries arrive; demand is drawn and adjusted for weekday and month; sales are made, with any shortfall a lost sale carried as backlog and penalized; orders are placed (after demand, as a real buyer orders once the day is known); profit is counted. Leftover stock at the end is valued at 60% of cost, and storage costs 0.15% of item cost per day. The suppliers (Table 1) are invented, since no public dataset records supplier terms; none dominates.

Decision and objective. Each day, for each product, one of $7 \times 3 = 21$ options is chosen: a quantity level and a supplier. Over 10 products this gives $21^{10} \approx 1.7 \times 10^{13}$ daily combinations. The goal is to maximize profit,

$$\begin{aligned} \text{profit} = \text{sales} - \text{purchases} - \text{storage} \\ - \text{fees} - \text{stockout} - \text{overflow}. \end{aligned} \tag{1}$$

Splits. Three disjoint seed groups are used: seeds 100–111 tune the classical policies, seeds 900–905 select checkpoints during training, and seeds 500–529 are used only for reporting. Every method faces the same customers and delivery delays on each reporting seed, so comparisons are paired.

Metrics. There is no ground-truth label in profit optimization, so classification accuracy is not defined. We report: (i) mean profit per 180-day run; (ii) relative profit, the agent's profit divided by the best classical profit; (iii) fill rate, the share of demand served; (iv) ordering, stockout, and overflow costs; and, for the explainer, (v) *raw accuracy* (grounded and complete) and *delivered accuracy* (after the check).

4 Method

4.1 System Pipeline

The system has five parts: the simulator; a MaskablePPO agent; four tuned classical policies; an LLM explainer; and an LLM crisis generator. A Streamlit dashboard presents them in shadow mode: the best classical policy is the *engine*, the agent runs alongside as a *challenger* that is scored but never acts, and every decision can be explained. The dashboard recommends and never places an order.

4.2 Agent and Objective

The observation has 88 numbers: for each product, stock on hand, stock on order, days of cover, the 7-day mean demand, demand variability, trend, and backlog; plus weekday, time of year, and each supplier's availability, lead time, and recent reliability. The action is a `MultiDiscrete` vector of ten entries over 21 joint options. Quantity and supplier are one joint choice, not two, which trains $2.8\times$ faster and reflects that ordering a large quantity is only sensible from a supplier able to deliver it. The reward is the day's profit, scaled and clipped, and zero during warm-up.

We use MaskablePPO. PPO maximizes the clipped surrogate

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{2}$$

with $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$. Masking adds a vector m to the logits z before the softmax,

$$\pi_\theta(a \mid s) = \text{softmax}(z+m)_a, \quad m_a = \begin{cases} 0 & \text{legal} \\ -\infty & \text{illegal,} \end{cases} \tag{3}$$

so offline suppliers and orders too large for the warehouse are never sampled. The network is small (two layers of 128 or 256 units) and trains on CPU.

4.3 Classical Baselines

Four textbook policies are implemented over the same 21 options per product as the agent, so none has finer control than another: (s, S), EOQ with a reorder point, the newsvendor model, and forecast plus safety stock. Each is tuned by grid search on the 12 tuning seeds. Tuning matters: widening the search lifted the best policy from Rs. 69.7 lakh to Rs. 89.2 lakh on the tuning seeds, and an untuned baseline would have made the agent look better than it is.

4.4 LLM Explainer with a Grounding Check

The LLM is *never asked why* a decision was made. The agent's real reason is a neural network, and any "because" the LLM wrote would be a plausible story rather than the cause. The LLM instead receives a set of facts F computed by the simulator (stock, orders in transit, days of cover, recent demand, what was ordered and from whom, that supplier's lead time and reliability) and writes two or three sentences. Let $N(x)$ denote the set of numbers appearing in text x. The output y is accepted only if $N(y) \subseteq N(F)$ and its required fields are present; otherwise a deterministic template is used. Three free models are tried in order (`gemma-4-31b-it`, `glm-5.2`, `nemotron-3-ultra-550b-a55b`) through OpenRouter, with retry and back-off because free endpoints rate-limit.

Setting	Value
Learning rate	5×10^{-5} (lowest offered)
Entropy coefficient	0.005
Discount γ	0.995
Rollout length	1024
Network	$[256, 256]$
Training steps	2M

Table 2: Winning hyperparameters of the 8-configuration search.

4.5 Crisis Generator and Curriculum

The LLM is asked for realistic supply-chain crises and returns scenarios such as a demand spike on four products, a supplier outage, a lead-time delay, and a cost surge, which overlap in time. Nothing it writes is trusted: the scenario type must be known, supplier and product indices must exist (out-of-range product numbers are dropped, not wrapped), values are clamped to safe ranges, and windows are moved inside the 180-day run. Six deliberately malformed inputs were all corrected or rejected.

For curriculum training, 16 crisis sets built from 19 scenarios were saved and split 10/6. The agent trains on a random training set at the start of each run and is tested on the 6 it never saw. Its seed, settings, and training length match the untuned agent, so crisis exposure is the only difference.

5 Experimental Setup

Training. All agents use MaskablePPO. The untuned agent trains for 1M steps with default settings. For the hyperparameter search, eight configurations received a short 250,000-step trial, and the best two were trained to 2M steps on Kaggle (about 4.8 hours). The winning configuration is in Table 2. Checkpoints are chosen on seeds 900–905 and results are reported on seeds 500–529.

The search chose the lowest learning rate and the larger network, which supports the diagnosis that earlier runs were unstable because they learned too fast, not because the network was too small.

Statistics. We use paired t-tests on per-seed differences ($n = 30$), and report the 95% interval of the mean difference.

6 Results

6.1 Main Comparison

Table 3 and Figure 1 compare all methods on the 30 held-out seeds. The best classical policy, forecast plus safety stock, earns Rs. 87.8 lakh. The tuned agent earns Rs. 79.7 lakh, which is **90.8%** of it. The difference is Rs. 8.1 lakh (paired $t = -6.45$, $n = 30$, $p < 0.001$, 95% CI [5.5, 10.6] lakh); the classical policy wins on 26 of 30 seeds. The agent beats two of the four classical policies, EOQ and (s, S). Against a fixed

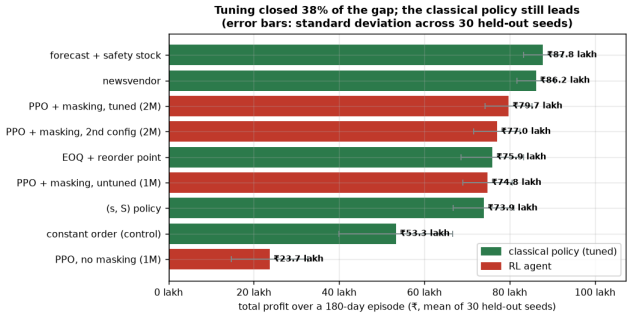


Figure 1: Profit of every method on the 30 held-out seeds. Green: classical; red: RL. Error bars are standard deviation across seeds.

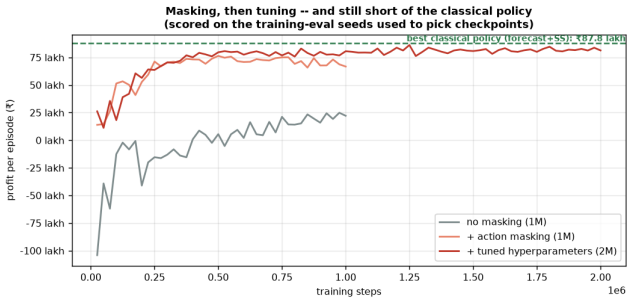


Figure 2: Learning progress on the checkpoint-selection seeds. This is a picture of learning, not a result; reported scores use held-out seeds.

daily order, which is what a shop does without any system, the engine raises profit from Rs. 53.3 lakh to Rs. 87.8 lakh (+65%) and wins on all 30 seeds; the agent also beats it on 29 of 30.

6.2 Ablation: What Each Step Was Worth

Table 4 isolates the two improvements. Action masking is the largest single lever (about 3×): without it, the agent spends most of training discovering that some choices are pointless. Tuning and longer training then close 38% of the remaining gap, from Rs. 13.0 lakh to Rs. 8.1 lakh. Figure 2 shows the learning curves.

6.3 Robustness to LLM-Generated Crises

Table 5 scores each method on 10 calm seeds and on 6 held-out crisis sets the agents never trained on. Every method loses more than a third of its profit. The tuned agent, which never saw a crisis in training, has the highest crisis profit and the smallest drop of all agent versions.

Curriculum training did not work. The crisis-trained agent earns Rs. 38.9 lakh under crisis against the control's Rs. 38.8 lakh, and gave up Rs. 7.7 lakh of calm profit. Its smaller percentage drop is misleading: it drops less because it has less to lose. The paired difference is not significant ($t = +1.69$, $n = 30$). We therefore show the percentage drop beside absolute profit, so that "less to lose" cannot pass for robustness.

6.4 Explainer Accuracy

We scored 30 real decisions (6 days × 10 products) (Table 6). The LLM was reachable on every call after retries. Five of 30 raw outputs contained a number the simulator never produced; they read fluently and a buyer would not have noticed. The check caught all five and substituted the template, so every explanation shown was correct and complete. Without the check, one explanation in six would carry a made-up figure.

An earlier version of the metric scored only outputs that had already passed the check and so reported 100% for a model that invented numbers 17% of the time. That value was true and uninformative; we report both.

6.5 Is the Data Too Old?

The simulator uses the *shape* of demand, not prices or volumes, so the relevant question is whether that shape is a feature of retail or of one period. Splitting the two years into two halves and measuring each (Figure 3), the weekday pattern matches strongly ($r = 0.87$) and November is the peak month in both. The month pattern is weaker ($r = 0.51$, or 0.64 excluding December, which is partly a data-window artifact), and burstiness (variance over mean) is extreme in both years (224 and 474). This supports the two features the simulator relies on most. It cannot speak to demand today, which stays a limitation (Section 9).

7 Error Analysis

Where does the Rs. 8.1 lakh gap come from? Figure 4 shows one traced run; the aggregate numbers in Table 3 agree.

- **Ordering fees.** The agent pays Rs. 5.8 lakh against Rs. 2.9 lakh. It spreads orders across all three suppliers: in the traced run it placed 719 orders, the classical policy 323, almost all with the cheap supplier.
- **Fill rate.** 95.1% against 97.9%; the agent runs out more often, costing about Rs. 3 lakh more in stockout penalties.
- **Overflow.** The agents are the only methods with zero overflow loss: they learned the shared-warehouse rule. Every classical policy wastes some.

The cause of the fee gap is an observability gap. The 88 inputs contain nothing about the agent's own ordering history: no days since the last order, no running total of fees, no record of the last supplier used. It pays double and cannot see that it does. This is a limit on what the agent observes, not a tuning problem, and adding a few history features per product is the most promising untried change; we have not tested it.

Method	Profit (Rs. lakh)	Std	Fill rate	Fees (Rs. lakh)	Stockout (Rs. lakh)
Forecast + safety stock	**87.8**	4.6	97.9%	2.9	6.1
Newsvendor	86.2	4.4	98.0%	3.4	7.4
RL agent, tuned, 2M steps	**79.7**	5.5	95.1%	5.8	9.2
RL agent, 2nd config, 2M steps	77.0	5.4	93.6%	7.1	10.5
EOQ + reorder point	75.9	7.4	96.8%	2.8	10.5
RL agent, untuned, 1M steps	74.8	5.8	94.3%	7.4	11.0
(s, S) policy	73.9	7.1	94.9%	3.1	16.5
Constant order (no system)	53.3	13.4	92.3%	3.5	29.0
RL agent, no masking, 1M steps	23.7	9.0	94.8%	14.7	11.3

Table 3: Mean profit per 180-day run over 30 held-out seeds (500–529). Std is across seeds. All methods face identical customers and delays on each seed. Rupee values are converted at 1 GBP = Rs. 105.

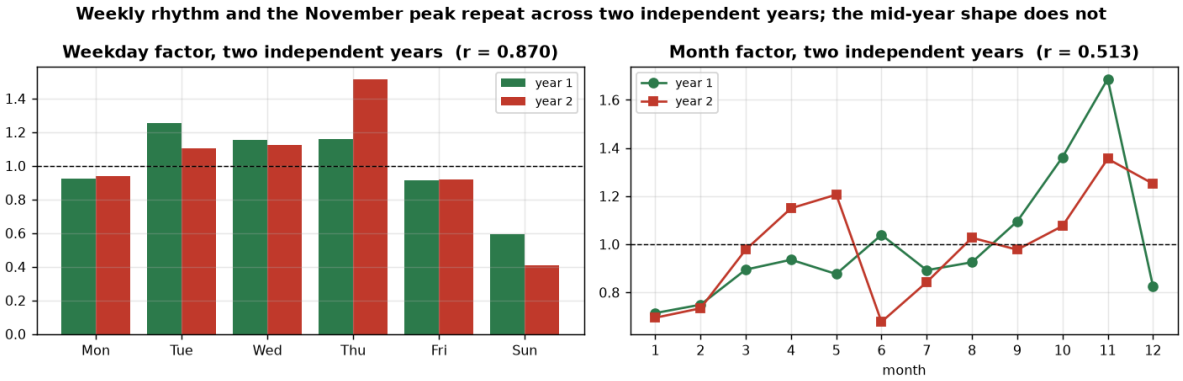


Figure 3: Demand pattern in year 1 (December 2009–November 2010) against year 2 (December 2010–November 2011).

Step	Train	Test	Gap
PPO, no masking	24.9	23.7	−64.1
+ action masking	76.4	74.8	−13.0
+ tuning, 2M steps	86.2	79.7	**−8.1**

Table 4: Profit in Rs. lakh. Train: checkpoint-selection seeds (900–905); Test: held-out seeds (500–529); Gap: test profit minus the best classical policy.

Method	Calm	Crisis	Drop
Forecast + safety stock	87.4	53.7	−38.6%
RL, tuned	80.4	**50.7**	**−37.0%**
RL, untuned (control)	73.8	38.8	−47.5%
RL, crisis-trained	66.0	38.9	−41.1%

Table 5: Profit in Rs. lakh on calm seeds and on 6 unseen LLM-generated crises.

8 Efficiency and Deployment

Everything trains on commodity hardware. A 1M-step run takes about 48 minutes on a laptop CPU; the 2M-step runs took about 4.8 hours on Kaggle. Curriculum training is roughly twice as expensive per step as calm training (measured at 156 steps per second). We did not measure inference latency or memory.

The result argues for a shadow-mode deployment, which the dashboard implements: the tuned classical policy is the engine a buyer would run, the agent runs beside it, logged and scored but not acting, and the explainer covers whichever decision is shown. When the challenger beats the engine consistently on live data it can be promoted. Still missing for real use: handling new products with no history, order-value limits, an audit trail, a live data feed, and a signal that a human should decide when the system is unsure.

Measure	Result
LLM reachable (after retries)	100%
Grounded (raw output)	83% (25/30)
Complete	100%
Raw model accuracy	**83%**
Delivered accuracy	**100%**
Length	2.9 sentences, 58 words

Table 6: Explainer evaluation on 30 decisions. Raw accuracy is grounded and complete before the check; delivered accuracy is what the user saw.

9 Limitations

We list these in descending order of how much they should change a reader's confidence.

Data age. The data is UK retail from 2009–2011. We showed the demand shape is stable across the two years we

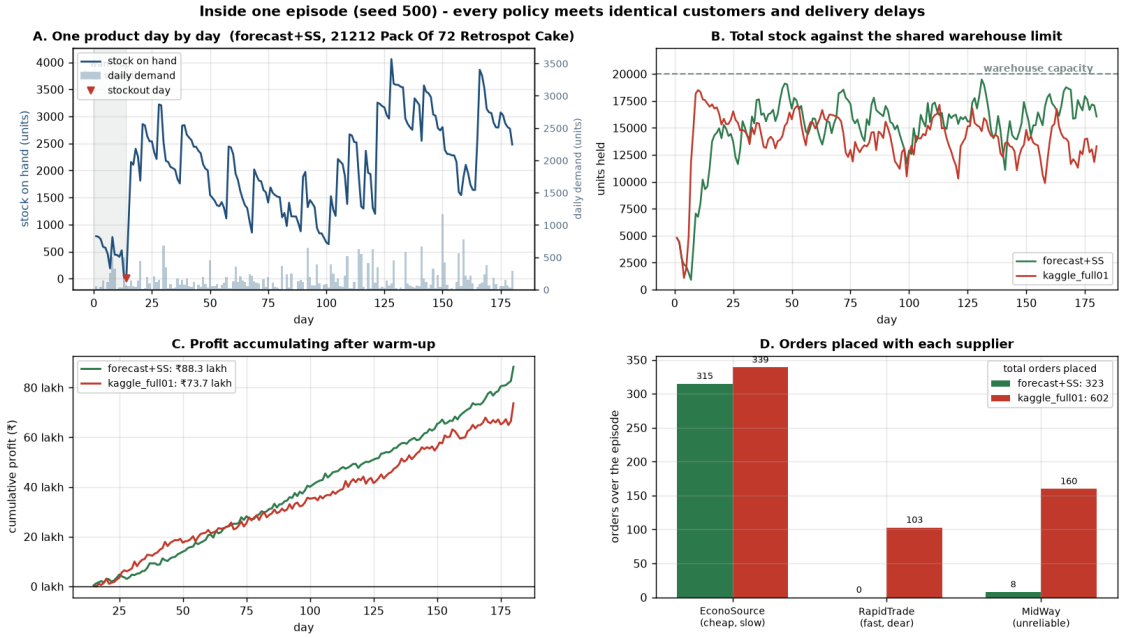


Figure 4: One run in the simulator: stock against demand for one product, warehouse fill, cumulative profit, and the supplier mix (panel D).

have, but nothing here speaks to current demand, and recent product-level retail data is not public.

Invented suppliers. Lead times, reliability, and fees are hand-set to span the standard trade-off. No public dataset records them, so absolute rupee values are not forecasts.

Unequal difficulty in curriculum training. The crisis-trained agent faced a harder task with the same step budget, so part of its weaker result may be under-training rather than the method failing.

Explainer sample. 30 decisions, one method, one seed. The explainer is a checked restatement of computed facts, not an account of the network's reasoning, and we do not claim otherwise.

One shop, ten products. Other categories, larger assortments, and other cost structures are untested.

Free LLM endpoints. Free models rate-limit; explanations can take up to a minute and sometimes fall back to the template.

10 Broader Impact

The agent should not be deployed today: it earns less than a simpler method and is not more robust. The honest use is as a scored challenger. Two risks deserve naming. An explanation that reads smoothly can lend unwarranted confidence to a wrong decision, which is why the numbers are checked and why the explanation is described as a restatement rather than a reason. And because suppliers are invented and the data is old, rupee figures should be read as relative comparisons inside one simulator.

11 Conclusion

A MaskablePPO agent reaches 90.8% of the profit of a tuned classical policy on 30 held-out seeds, beats two of four classical policies, and beats a fixed daily order on 29 of 30 seeds, but the best classical policy still wins by Rs. 8.1 lakh per run ($t = -6.45$). The result is a negative one for the hypothesis that RL adapts better than a tuned formula: it did not, in calm conditions or in crisis. We think the part worth keeping is how the result was established. The baselines were tuned, the test seeds were separate, the crisis-trained agent was tested on unseen crises, and the percentage drop was shown beside absolute profit. Masking and tuning each had a measurable effect; crisis training did not. The explainer is accurate only because it is checked: 83% raw, 100% delivered.

The clearest next step is indicated by the error analysis. The gap is fees from supplier fragmentation, and the agent cannot see its own ordering history, so giving it that history while keeping the same evaluation is the experiment most likely to change the headline result.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms.

arXiv:1707.06347, 2017.

[2] S. Huang and S. Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *FLAIRS-35*, 2022.

[3] J. Gijsbrechts, R. N. Boute, J. A. Van Mieghem, and D. Zhang. Can deep reinforcement learning improve inventory management? *Manufacturing & Service Operations Management*, 2022.

[4] R. N. Boute, J. Gijsbrechts, W. van Jaarsveld, and N. Vanvuchelen. Deep reinforcement learning for inventory control: A roadmap. *European Journal of Operational Research*, 2022.

[5] A. Oroojlooyjadid, M. Nazari, L. V. Snyder, and M. Takáč. A deep Q-network for the beer game. *Manufacturing & Service Operations Management*, 2022.

[6] D. Chen, S. L. Sain, and K. Guo. Data mining for the online retail industry. *Journal of Database Marketing & Customer Strategy Management*, 2012.

[7] A. Raffin et al. Stable-Baselines3: Reliable reinforcement learning implementations. *JMLR*, 22(268), 2021.

[8] UCI Machine Learning Repository. Online Retail II. `https://archive.ics.uci.edu/dataset/502`.